Manual de Usuario — Nexus AILang-Core en Visual Studio Code

Versión: 1.1.0 **Autor:** Enrique Jimenez Garcia

Tabla de Contenidos

1. Instalación
2. Instalar la Extensión de VS Code
3. Crear tu Primer Proyecto
4. Escribir Código AILang
5. Ejecutar Programas
6. Depurar con Breakpoints
7. Análisis de Capabilities
8. Formatear Código
9. Usar el Linter
10. Language Server Protocol (LSP)
11. Crear un Paquete
12. Playground Web
13. Solución de Problemas

---

1. Instalación

Requisitos previos

- **Python 3.10 o superior**
- **Visual Studio Code** (versión 1.75 o superior)
- **pip** (incluido con Python)

Instalar Nexus AILang-Core

Abre una terminal y ejecuta:

```
pip install nexus-ailang-core
```

Instalar dependencias opcionales (recomendado)

```
pip install nexus-ailang-core[crypto]    # Para crypto (Ed25519, AES-256-GCM)
pip install nexus-ailang-core[postgres]  # Para PostgreSQL
pip install nexus-ailang-core[llvm]      # Para LLVM JIT backend
```

```
pip install nexus-ailang-core[testing]  # Para hypothesis + pytest
pip install nexus-ailang-core[all]      # Todas las anteriores
```

Verificar la instalación

```
python -m ailang --version    # Debe mostrar: AILang 1.1.0
python -m ailang doctor       # Debe mostrar: 11/11 componentes OK
```

---

2. Instalar la Extensión de VS Code

Opción A: Instalación manual

1. Abre VS Code
2. Ve a la carpeta `vscode-ailang/` dentro del repositorio
3. Ejecuta en terminal:

```
cd vscode-ailang
code --install-extension .
```

Opción B: Empaquetar e instalar

```
npm install -g @vscode/vsce
cd vscode-ailang
vsce package
code --install-extension ailang-1.1.0.vsix
```

Verificar la extensión

1. Abre VS Code
2. Crea un archivo llamado `test.ailang`
3. Escribe `print("hello")` — debes ver syntax highlighting (colores)

---

3. Crear tu Primer Proyecto

Desde la terminal

```
python -m ailang new app mi-primera-app
cd mi-primera-app
code .
```

Desde VS Code

1. Abre VS Code

2. Presiona `Ctrl+Shift+P` (Windows/Linux) o `Cmd+Shift+P` (Mac)
3. Escribe `AILang: New Project` y presiona Enter
4. Escribe el nombre del proyecto (ej: `mi-app`)
5. Selecciona el tipo: `app`, `lib`, `web`, o `script`
6. Selecciona la carpeta donde crear el proyecto

Estructura generada (template `app`)

```
mi-primera-app/
├── .github/
│   └── workflows/
│       └── ci.yml          # CI preconfigurado
├── tests/
│   └── test_main.py        # Test inicial
├── ailang.toml              # Manifest del proyecto
├── main.ailang              # Entry point
└── README.md                # Documentación
```

Ejecutar el proyecto generado

```
python -m ailang main.ailang
```

Salida esperada:

```
Hello from mi-primera-app!
Running on AILang vailang-1.1.0
```

---

4. Escribir Código AILang

Sintaxis básica

AILang tiene una sintaxis familiar, similar a Python y JavaScript:

```
// Variables
let nombre = "Ada"
let edad = 30
let pi = 3.14
let activo = true
let nada = null

// Funciones
fn saludar(nombre) {
  return "Hola " + nombre
}

// Llamar función
print(saludar("Mundo"))  // Hola Mundo
```

Snippets disponibles en VS Code

Escribe el prefijo y presiona `Tab` para expandir:

Prefijo	Genera	Descripción
`fn`	Función completa	Declaración de función
`afn`	Async function	Función asíncrona
`class`	Clase con init	Clase con constructor
`record`	Record	Tipo producto
`enum`	Enumeración	Tipo enumerado
`interface`	Interfaz	Contrato de métodos
`if`	If statement	Condicional
`ife`	If-else	Condicional con else
`while`	While loop	Bucle while
`for`	For loop	Bucle for-in
`match`	Match expression	Pattern matching
`try`	Try-catch	Manejo de excepciones
`import`	Import	Importar módulo
`gen`	Generador	Función con yield
`chan`	Channel	Canal send/recv
`spawn`	Spawn	Lanzar tarea
`let`	Variable	Declaración de variable
`const`	Constante	Declaración de constante

Ejemplo completo

Crea un archivo `demo.ailang` y pega:

```
// Variables con type inference
let nombre = "Ada"
let edad = 30

// Función con closures mutables
fn make_counter() {
  let count = 0
  fn inc() {
    count = count + 1
    return count
  }
  return inc
}

// Clases con herencia
class Animal {
  fn init(name) { self.name = name }
  fn speak() { return "sonido genérico" }
}

class Perro extends Animal {
  fn speak() { return "guau" }
}

// Generadores
fn gen_numeros() {
  yield 1
  yield 2
  yield 3
}

// Usar todo
let contador = make_counter()
print(contador())  // 1
print(contador())  // 2

let mi_perro = Perro("Rex")
print(mi_perro.name)       // Rex
print(mi_perro.speak())    // guau
```

```
let total = 0
for n in gen_numeros() {
  total = total + n
}
print("total: " + str(total))  // total: 6

// String interpolation
print("Hola ${nombre}, edad ${edad}")
```

---

5. Ejecutar Programas

Método 1: Comando de VS Code

1. Abre un archivo `.ailang`
2. Presiona `Ctrl+Shift+P` o `Cmd+Shift+P`
3. Escribe uno de estos comandos:
   - `AILang: Run File` — ejecuta con el backend por defecto
   - `AILang: Run File (VM)` — ejecuta con VM (más rápido)
   - `AILang: Run File (AIR + Capabilities)` — verifica capabilities antes de ejecutar

Método 2: Click derecho

1. Click derecho en el editor
2. Selecciona `AILang: Run File`

Método 3: Botón Run

1. Abre un archivo `.ailang`
2. Click en el botón ▶ Play en la barra del editor

Método 4: Terminal

```bash
# Interpreter (default)
python -m ailang archivo.ailang

# VM (más rápido, 1.5-2.3x)
python -m ailang --vm archivo.ailang

# AIR + capability verification
python -m ailang --air archivo.ailang --allow filesystem.read

# Ejecutar string inline
python -m ailang -c "1 + 2 * 3"
```

Configurar el backend por defecto

1. Ve a `File → Preferences → Settings` (o `Code → Settings`)
2. Busca `ailang`
3. En `ailang.defaultBackend`, selecciona:
   - `interp` — Interpreter (más rápido al iniciar)

- `vm` — VM (más rápido en ejecución, recomendado)
- `air` — AIR (verifica capabilities antes de ejecutar)

---

6. Depurar con Breakpoints

Configurar el debugger

1. Abre un archivo `.ailang`
2. Ve a la pestaña `Run and Debug` (o presiona `Ctrl+Shift+D`)
3. Click en `create a launch.json file`
4. Selecciona `AILang Debugger`

Se creará `.vscode/launch.json`:

```
{
  "type": "ailang",
  "request": "launch",
  "name": "AILang: Debug",
  "program": "${file}"
}
```

Establecer breakpoints

1. Abre un archivo `.ailang`
2. Click en el margen izquierdo (gutter) junto al número de línea
3. Aparece un punto rojo — el breakpoint está activo

Controles de depuración

Tecla	Acción	Descripción
`F5`	Continue	Ejecuta hasta el próximo breakpoint
`F10`	Step Over	Ejecuta la siguiente línea (sin entrar en funciones)
`F11`	Step Into	Entra en la función llamada
`Shift+F11`	Step Out	Sale de la función actual
`Shift+F5`	Stop	Detiene la ejecución

Inspeccionar variables

Durante la depuración, en el panel izquierdo:

- **Variables → Locals**: variables locales del scope actual
- **Variables → Globals**: variables globales
- **Watch**: añade expresiones para monitorear

Evaluación de expresiones

En la `Debug Console` (panel inferior), escribe:

```
nombre        // evalúa la variable 'nombre'
1 + 2         // evalúa expresión
str(edad)     // convierte a string
```

---

7. Análisis de Capabilities

Verificar capabilities de un programa

```
python -m ailang check archivo.ailang
```

Salida de ejemplo:

```
{
  "bytecode_version": 7,
  "language_version": "1.1.0",
  "functions": [
    {
      "name": "leer_config",
      "params": ["path"],
      "capabilities": ["filesystem.read", "filesystem.write"],
      "effects": ["filesystem"]
    },
    {
      "name": "consultar_brain",
      "params": ["text"],
      "capabilities": ["ai.query"],
      "effects": ["ai"]
    }
  ],
  "required_capabilities": ["ai.query", "filesystem.read", "filesystem.write"],
  "required_effects": ["ai", "filesystem"]
}
```

Ejecutar con capabilities específicas

```
# Denegar todo por defecto, permitir solo lo necesario
python -m ailang archivo.ailang --allow filesystem.read --allow ai.query

# Usar wildcards
python -m ailang archivo.ailang --allow filesystem.*

# Denegar explícitamente una capability
python -m ailang archivo.ailang --allow filesystem.* --deny filesystem.write
```

Capabilities disponibles

Capability	Operaciones que permite
`filesystem.read`	Leer archivos
`filesystem.write`	Escribir archivos
`network.http`	Requests HTTP
`process.spawn`	Crear procesos/hilos
`ai.query`	Operaciones cognitivas (brain.*)
`hardware.gpu`	Usar GPU
`hardware.npu`	Usar NPU

Capability	Operaciones que permite
`hardware.tpu`	Usar TPU
`database.read`	Leer base de datos
`database.write`	Escribir base de datos
`system.env`	Variables de entorno
`crypto.use`	Operaciones criptográficas

---

8. Formatear Código

Formatear archivo actual

1. Abre un archivo `.ailang`
2. Presiona `Ctrl+Shift+P` o `Cmd+Shift+P`
3. Escribe `AILang: Format File` y presiona Enter

Formatear desde terminal

```
python -m ailang fmt archivo.ailang
```

Formatear al guardar

1. Ve a `File → Preferences → Settings`
2. Busca `ailang`
3. Activa `ailang.formatOnSave`

Cada vez que guardes un archivo `.ailang`, se formateará automáticamente.

El formatter es idempotente

El formatter AST-based garantiza que formatear dos veces produce el mismo resultado:

```
python -m ailang fmt archivo.ailang   # primera vez
python -m ailang fmt archivo.ailang   # segunda vez = sin cambios
```

---

9. Usar el Linter

Ejecutar el linter

1. Abre un archivo `.ailang`
2. Presiona `Ctrl+Shift+P` o `Cmd+Shift+P`

3. Escribe `AILang: Check (Analysis)` — NO, usa `AILang: Format File`

O desde terminal:

```
python -m ailang lint archivo.ailang
```

Reglas del linter

Código	Nivel	Descripción
E001	warning	Variable asignada pero nunca usada
E002	warning	Import nunca usado
W001	info	Shadowing de variable
W002	warning	Código inalcanzable después de return/break/continue
W003	info	Función podría no retornar en todos los paths
W005	warning	Bloque vacío (if/while/fn sin body)

Ejemplo

```
let x = 1
let unused = 2    // E001: variable 'unused' asignada pero nunca usada
import json        // E002: import 'json' nunca usado
print(x)
```

---

10. Language Server Protocol (LSP)

El LSP proporciona inteligencia de código en VS Code:

Features disponibles

Feature	Cómo usar
Autocompletado	Escribe y aparecerán sugerencias
Hover	Pasa el cursor sobre un identificador
Go to Definition	`Ctrl+Click` o `F12` sobre un nombre
References	`Shift+F12` sobre un identificador
Rename	`F2` sobre un identificador
Document Symbols	`Ctrl+Shift+O` para ver outline
Semantic Tokens	Colores semánticos (keywords, tipos, funciones)
Signature Help	Al escribir `(` muestra parámetros de la función
Diagnostics	Errores de sintaxis en tiempo real

Habilitar/deshabilitar LSP

1. Ve a `File → Preferences → Settings`
2. Busca `ailang`

3. Activa/desactiva `ailang.lspEnabled`

---

11. Crear un Paquete

Inicializar un paquete

```
python -m ailang pkg init mi-libreria
```

Instalar un paquete local

```
python -m ailang pkg install ruta/al/paquete
```

Publicar un paquete (crear .ailpkg)

```
python -m ailang pkg publish mi-libreria --sign
```

Esto genera `~/.ailang/packages/registry/mi-libreria-0.1.0.ailpkg` con:

- `manifest.json` — metadata del paquete
- Archivos del paquete
- `checksum.sha256` — SHA-256 de cada archivo
- `signature.bin` — firma Ed25519 (si se usó `--sign`)

Instalar desde .ailpkg

```
python -m ailang pkg install mi-libreria-0.1.0.ailpkg
```

Buscar paquetes

```
python -m ailang pkg search query
```

Listar paquetes instalados

```
python -m ailang pkg list
```

Eliminar un paquete

```
python -m ailang pkg remove mi-libreria
```

---

12. Playground Web

Iniciar el playground

```
python -m ailang --serve
```

Abre tu navegador en `http://localhost:8080`

Features del playground

- **Editor con syntax highlighting** (CodeMirror)
- **13 ejemplos pre-cargados**: Hello World, Fibonacci, Closures, Classes, Generators, Channels, Records, Enums, Try-Catch, Strings, Crypto, Tensor, Capabilities
- **Selector de backend**: Interpreter, VM, AIR
- **Botón Format**: formatea el código
- **Output panel**: muestra stdout, stderr, y resultado
- **Status bar**: tiempo de ejecución y backend usado

Cambiar puerto

```
python -m ailang --serve --port 3000
```

---

13. Solución de Problemas

"AILang no reconocido como comando"

```
# Verificar instalación
pip show nexus-ailang-core

# Reinstalar si es necesario
pip install --force-reinstall nexus-ailang-core
```

"python -m ailang doctor muestra errores"

```
# Instalar dependencias faltantes
pip install numpy cryptography hypothesis pytest

# Verificar Python versión
python --version  # Debe ser 3.10+
```

"VS Code no reconoce archivos .ailang"

1. Abre un archivo `.ailang`
2. Click en el selector de lenguaje (esquina inferior derecha)
3. Busca "AILang" en la lista

4. Selecciona "AILang"

"El debugger no se detiene en breakpoints"

1. Verifica que el breakpoint está activo (punto rojo sólido)
2. Asegúrate de usar `AILang: Debug` en launch.json
3. Verifica que `ailang.dapEnabled` está activado en Settings
4. Intenta usar `python -m ailang --debug archivo.ailang --bp 5` desde terminal

"El formatter cambia mi código de forma inesperada"

El formatter AST-based normaliza:

- Indentación a 2 espacios
- Llaves en la misma línea del header
- Espacios alrededor de operadores
- Una expresión por línea

Si no quieres formatear, usa `format_source` (token-based) en lugar de `format_ast`:

```
python -c "from ailang.fmt import format_source;
        print(format_source(open('archivo.ailang').read()))"
```

"Capabilities denegadas al ejecutar"

```
# Ver qué capabilities necesita el programa
python -m ailang check archivo.ailang

# Conceder las capabilities necesarias
python -m ailang archivo.ailang --allow filesystem.read --allow ai.query
```

Configurar Python path en VS Code

Si VS Code no encuentra Python:

1. Ve a `File → Preferences → Settings`
2. Busca `ailang.pythonPath`
3. Escribe la ruta completa a tu Python:
   - Linux/Mac: `/usr/bin/python3` o `/usr/local/bin/python3`
   - Windows: `C:\\Python312\\python.exe` o la ruta de tu virtualenv

Apéndice A: Módulos Stdlib

Módulo	Funciones principales
`io`	read_text, write_text, exists, lines
`math`	abs, min, max, sqrt, pow, floor, ceil, pi, e
`strings`	upper, lower, contains, split, replace
`json`	loads, dumps
`time`	now, sleep, format_ts, monotonic
`http`	get, post, json_get
`path`	join, exists, is_file, is_dir, mkdir, listdir
`sys`	argv, exit, getenv, platform, version
`random`	seed, random, randint, choice, shuffle
`listops`	map, filter, reduce, sum, sorted
`test`	assert_true, assert_eq, assert_raises, summary
`re`	search, match, findall, sub, split
`brain`	ingest, query, think, graph
`crypto`	sha256, sha512, blake2b, hmac, sign, verify, aes_encrypt, aes_decrypt
`tensor`	tensor, zeros, ones, randn, eye, matmul, softmax, relu, backward

Apéndice B: Comandos CLI

```
python -m ailang file.ailang              # Ejecutar archivo
python -m ailang --vm file.ailang          # Ejecutar con VM
python -m ailang --air file.ailang         # Ejecutar con AIR
python -m ailang -c "1 + 2"               # Ejecutar string
python -m ailang --repl                    # REPL interactivo
python -m ailang check file.ailang         # Análisis sin ejecución
python -m ailang fmt file.ailang           # Formatear
python -m ailang lint file.ailang          # Linter
python -m ailang bench                     # Benchmarks
python -m ailang doctor                    # Diagnóstico
python -m ailang --serve                   # Playground web
python -m ailang --lsp                     # Language Server
python -m ailang --dap                     # Debug Adapter
python -m ailang --version                 # Versión
python -m ailang new app myapp             # Crear proyecto
python -m ailang pkg init/install/list/run/publish/update/remove/search
```

Apéndice C: Configuración de VS Code

settings.json recomendado

```
{
  "ailang.pythonPath": "python3",
  "ailang.lspEnabled": true,
  "ailang.dapEnabled": true,
  "ailang.formatOnSave": true,
  "ailang.defaultBackend": "vm",
  "ailang.capabilities": ["filesystem.read", "ai.query"],
  "editor.tabSize": 2,
  "files.associations": {
    "*.ailang": "ailang",
    "*.ail": "ailang"
  }
}
```

launch.json para debugging

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "ailang",
      "request": "launch",
      "name": "AILang: Debug",
      "program": "${file}"
    },
    {
      "type": "ailang",
      "request": "launch",
      "name": "AILang: Debug (VM)",
      "program": "${file}",
      "backend": "vm"
    }
  ]
}
```

Manual de Usuario — Nexus AILang-Core v1.1.0 Autor: Enrique Jimenez Garcia **Licencia:** MIT **Repo:** https://github.com/Nexusolutions/nexus-ailang-core